

Misión Órbita — Avance 1

Informe académico de pruebas y documentación de programas en Python

Ficha del informe

Campo	Información
Repositorio GitHub:	ENLACE
Curso	SOFT-01 Principios de Programación 1
Sección	SCV3
Docente	Verónica Mora Lezcano
Estudiante	Dylan Garos Quesada
Fecha	9 de octubre de 2026
Versión	1.0

Nota

Documento preparado para documentar la implementación, las pruebas y las evidencias de cuatro ejercicios de programación.

1. Introducción y objetivo

El proyecto Misión Órbita consiste en desarrollar cuatro programas en Python para apoyar la planificación y evaluación de una misión espacial. Los programas permiten calcular los recursos necesarios, verificar la disponibilidad de combustible, comprobar si existen suficientes recursos para la tripulación y determinar si se cumplen las condiciones para autorizar un lanzamiento.

Para resolver los ejercicios se utilizan variables, constantes, operaciones aritméticas y estructuras condicionales, siguiendo las restricciones establecidas para el avance.

Objetivo general

Desarrollar cuatro programas en Python que permitan calcular y verificar los recursos necesarios para una misión espacial mediante estructuras secuenciales y condicionales.

Objetivos específicos

Calcular las necesidades de combustible, oxígeno, energía y provisiones.

Comparar el combustible disponible con el requerido y detectar márgenes bajos.

Verificar la disponibilidad de oxígeno y provisiones para la tripulación.

Determinar si una misión cumple las condiciones necesarias para autorizar su lanzamiento.

Documentar las soluciones mediante código, tablas de variables, diagramas de flujo y casos de prueba.

2. Matriz general de casos de prueba

ID	Ejercicio	Escenario	Resultado esperado	Resultado obtenido	Estado
CP01	1	Explorador I: 3 tripulantes, 5 días	Combustible: 90; oxígeno: 35; energía: 60; provisiones: 33	Ver captura CP01	Pendiente de registrar
CP02	1	Luna II: 2 tripulantes, 3 días	Combustible: 58; oxígeno: 17; energía: 40; provisiones: 9	Ver captura CP02	Pendiente de registrar
CP03	1	Marte III: 4 tripulantes, 10 días	Combustible: 170; oxígeno: 85; energía: 110; provisiones: 43	Ver captura CP03	Pendiente de registrar
CP01	2	Disponible: 100; ida: 35; regreso: 35	Total: 80; margen: 20; sin advertencia	Ver captura CP01	Pendiente de registrar
CP02	2	Disponible: 80; ida: 35; regreso: 35	Total: 80; margen: 0; con advertencia	Ver captura CP02	Pendiente de registrar
CP03	2	Disponible: 70; ida: 35; regreso: 35	Total: 80; margen: -10; con advertencia	Ver captura CP03	Pendiente de registrar
CP01	3	Oxígeno: 50/35; provisiones: 40/33	Recursos suficientes	Ver captura CP01	Pendiente de registrar
CP02	3	Oxígeno: 30/35; provisiones: 40/33	Recursos insuficientes	Ver captura CP02	Pendiente de registrar
CP03	3	Oxígeno: 40/35; provisiones: 30/33	Recursos insuficientes	Ver captura CP03	Pendiente de registrar

CP01	4	Todos los recursos suficientes	Lanzamiento autorizado	Ver captura CP01	Pendiente de registrar
CP02	4	Oxígeno disponible: 30; requerido: 35	Lanzamiento no autorizado	Ver captura CP02	Pendiente de registrar
CP03	4	Combustible disponible: 79; requerido: 80	Lanzamiento no autorizado	Ver captura CP03	Pendiente de registrar

3. Ejercicio 1 — Cálculo de recursos de misión

Documentación del cálculo de combustible, oxígeno, energía y provisiones a partir del número de tripulantes y la duración de la misión.

Código Python

==========

Declaración de constantes para los cálculos de recursos.

```
COMBUSTIBLE_POR_DIA = 8

OXIGENO_POR_TRIPULANTE_DIA = 2

OXIGENO_EMERGENCIA = 5

ENERGIA_POR_DIA = 5

ENERGIA_EXPLORACION = 10

PROVISIONES_POR_TRIPULANTE_DIA = 1

PROVISIONES_EMERGENCIA = 3

RESERVA_COMBUSTIBLE = 10

DIAS_POR_VIAJE = 2
```

Entrada de datos: se solicitan los datos básicos de la misión.

```
nombre_mision = input("Ingrese el nombre de la misión: ")

tripulantes = int(input("Ingrese la cantidad de tripulantes: "))

dias_viaje = int(input("Ingrese los días estimados para llegar al destino: "))
```

Procesamiento: se calculan los recursos para el viaje de ida y regreso.

```
combustible_ida = dias_viaje * COMBUSTIBLE_POR_DIA

combustible_regreso = dias_viaje * COMBUSTIBLE_POR_DIA

combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

oxigeno_requerido = (

dias_viaje * OXIGENO_POR_TRIPULANTE_DIA * tripulantes + OXIGENO_EMERGENCIA
```

```
)

energia_requerida = (

dias_viaje * ENERGIA_POR_DIA * DIAS_POR_VIAJE + ENERGIA_EXPLORACION

)

provisiones_requeridas = (

dias_viaje * tripulantes * DIAS_POR_VIAJE + PROVISIONES_EMERGENCIA

)
```

Salida: se presenta el resumen ordenado de los recursos necesarios.

```
print("\n========== RESUMEN DE LA MISIÓN ==========")

print(f"Misión: {nombre_mision}")

print(f"Tripulantes: {tripulantes}")

print(f"Duración estimada hasta el destino: {dias_viaje} días")

print(f"Combustible para llegar: {combustible_ida} unidades")

print(f"Combustible para regresar: {combustible_regreso} unidades")

print(f"Reserva de combustible: {RESERVA_COMBUSTIBLE} unidades")

print(f"Combustible total requerido: {combustible_total} unidades")

print(f"Oxígeno requerido: {oxigeno_requerido} unidades")

print(f"Energía requerida: {energia_requerida} unidades")

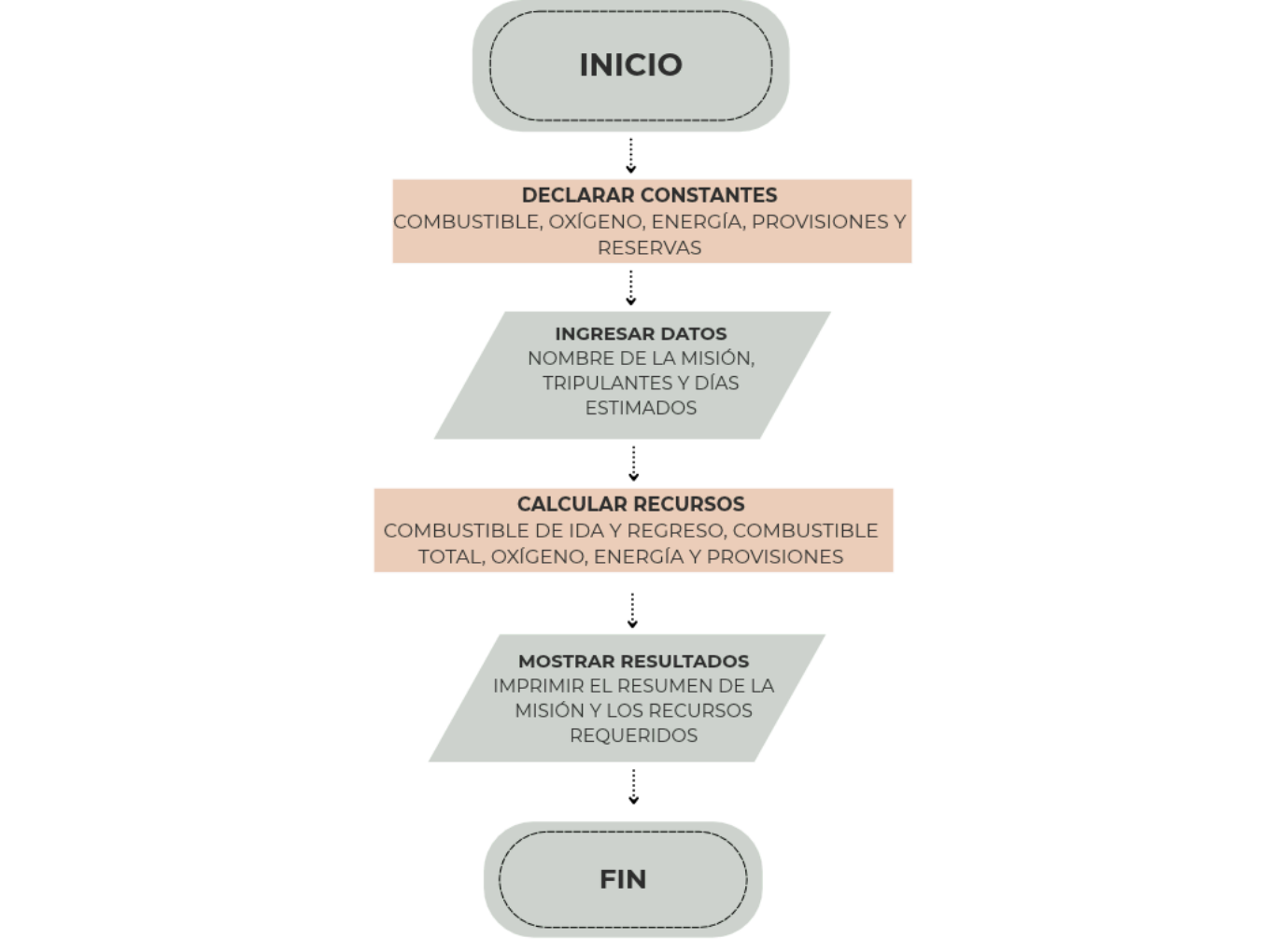
print(f"Provisiones requeridas: {provisiones_requeridas} unidades")
```

Tabla de variables

Variable	Descripción	Tipo	Valor o fuente
nombre_mision	Nombre de la misión	Cadena de texto	Entrada del usuario
tripulantes	Cantidad de tripulantes	Entero	Entrada del usuario
dias_viaje	Duración de la misión	Entero	Entrada del usuario
combustible_ida	Combustible requerido para la ida	Entero	Días × 8
combustible_regreso	Combustible requerido para el regreso	Entero	Días × 8
combustible_total	Combustible total con reserva	Entero	Ida + regreso + 10
oxigeno_requerido	Oxígeno requerido	Entero	Días × 2 × tripulantes + 5
energia_requerida	Energía requerida	Entero	Días × 5 × 2 + 10
provisiones_requeridas	Provisiones requeridas	Entero	Días × tripulantes × 1 + 3

Las constantes utilizadas en los cálculos deben documentarse con sus nombres exactos, según aparecen en el código fuente.

Diagrama de flujo



Evidencias

Evidencia CP01

Observación CP01: Se calculan los recursos para 3 tripulantes durante 5 días. Se espera un total de 90 unidades de combustible, 35 de oxígeno, 60 de energía y 33 de provisiones.

```
Ingrese el nombre de la misión: Explorador 1
Ingrese la cantidad de tripulantes: 3
Ingrese los días estimados para llegar al destino: 5

========== RESUMEN DE LA MISIÓN ==========
Misión: Explorador 1
Tripulantes: 3
Duración estimada hasta el destino: 5 días
Combustible para llegar: 40 unidades
Combustible para regresar: 40 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 90 unidades
Oxígeno requerido: 35 unidades
Energía requerida: 60 unidades
Provisiones requeridas: 33 unidades
```

Evidencia CP02

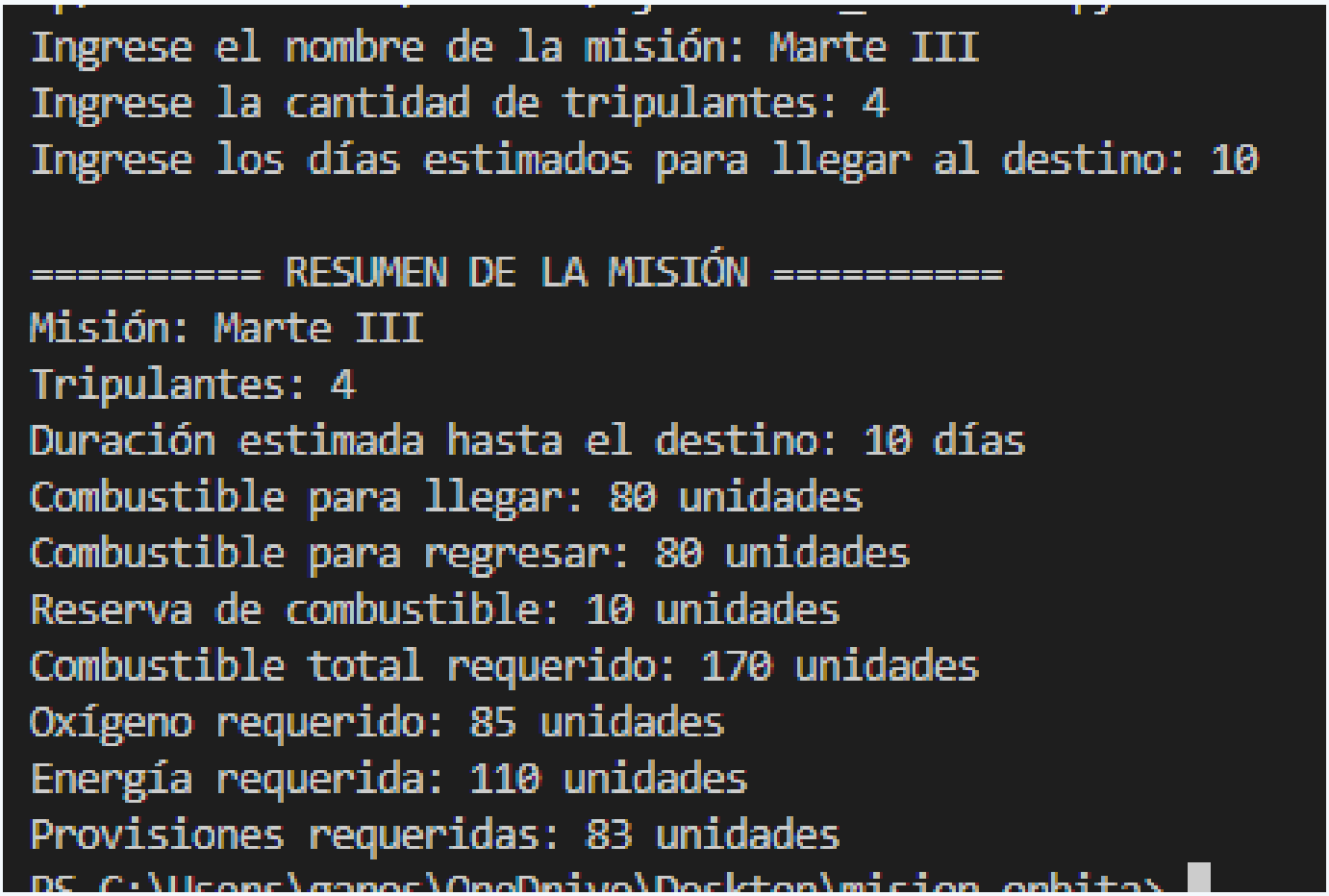
Observación CP02: Se calculan los recursos para 2 tripulantes durante 3 días. Se espera un total de 58 unidades de combustible, 17 de oxígeno, 40 de energía y 9 de provisiones.

```
Ingrese el nombre de la misión: Luna II
Ingrese la cantidad de tripulantes: 2
Ingrese los días estimados para llegar al destino: 3

========== RESUMEN DE LA MISIÓN ==========
Misión: Luna II
Tripulantes: 2
Duración estimada hasta el destino: 3 días
Combustible para llegar: 24 unidades
Combustible para regresar: 24 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 58 unidades
Oxígeno requerido: 17 unidades
Energía requerida: 40 unidades
Provisiones requeridas: 15 unidades
```

Evidencia CP03

Observación CP03: Se calculan los recursos para 4 tripulantes durante 10 días. Se espera un total de 170 unidades de combustible, 85 de oxígeno, 110 de energía y 43 de provisiones.



Observaciones

El programa solicita los datos de la misión y calcula secuencialmente el combustible para la ida y el regreso, las reservas y los recursos necesarios para la tripulación. Los resultados permiten estimar los recursos antes de realizar el viaje.

4. Ejercicio 2: código y variables

Documentación de la comparación entre combustible disponible, consumo de ida y consumo de regreso, incluyendo el margen y la advertencia esperada.

Código Python

Declaración de constantes para los cálculos.

```
RESERVA_COMBUSTIBLE = 10

MARGEN_MINIMO_COMBUSTIBLE = 10
```

Entrada de datos: solicitar las cantidades de combustible.

```
combustible_disponible = int(

input("Ingrese el combustible disponible: ")

)

combustible_ida = int(

input("Ingrese el combustible requerido para la ida: ")

)

combustible_regreso = int(

input("Ingrese el combustible requerido para el regreso: ")

)
```

Procesamiento: calcular el combustible total y el margen.

```
combustible_total_requerido = (

combustible_ida

+ combustible_regreso

+ RESERVA_COMBUSTIBLE

)

margen_adicional = (

combustible_disponible - combustible_total_requerido

)
```

Decisión: verificar si el margen adicional es bajo.

```
if margen_adicional < MARGEN_MINIMO_COMBUSTIBLE:

print(

"Advertencia: el margen adicional de combustible es bajo."

)
```

Salida: mostrar el resumen de combustible.

```
print("\n========== RESUMEN DE COMBUSTIBLE ==========")

print(f"Combustible disponible: {combustible_disponible} unidades")

print(f"Combustible para la ida: {combustible_ida} unidades")

print(f"Combustible para el regreso: {combustible_regreso} unidades")

print(f"Reserva de combustible: {RESERVA_COMBUSTIBLE} unidades")

print(

f"Combustible total requerido: "

f"{combustible_total_requerido} unidades"

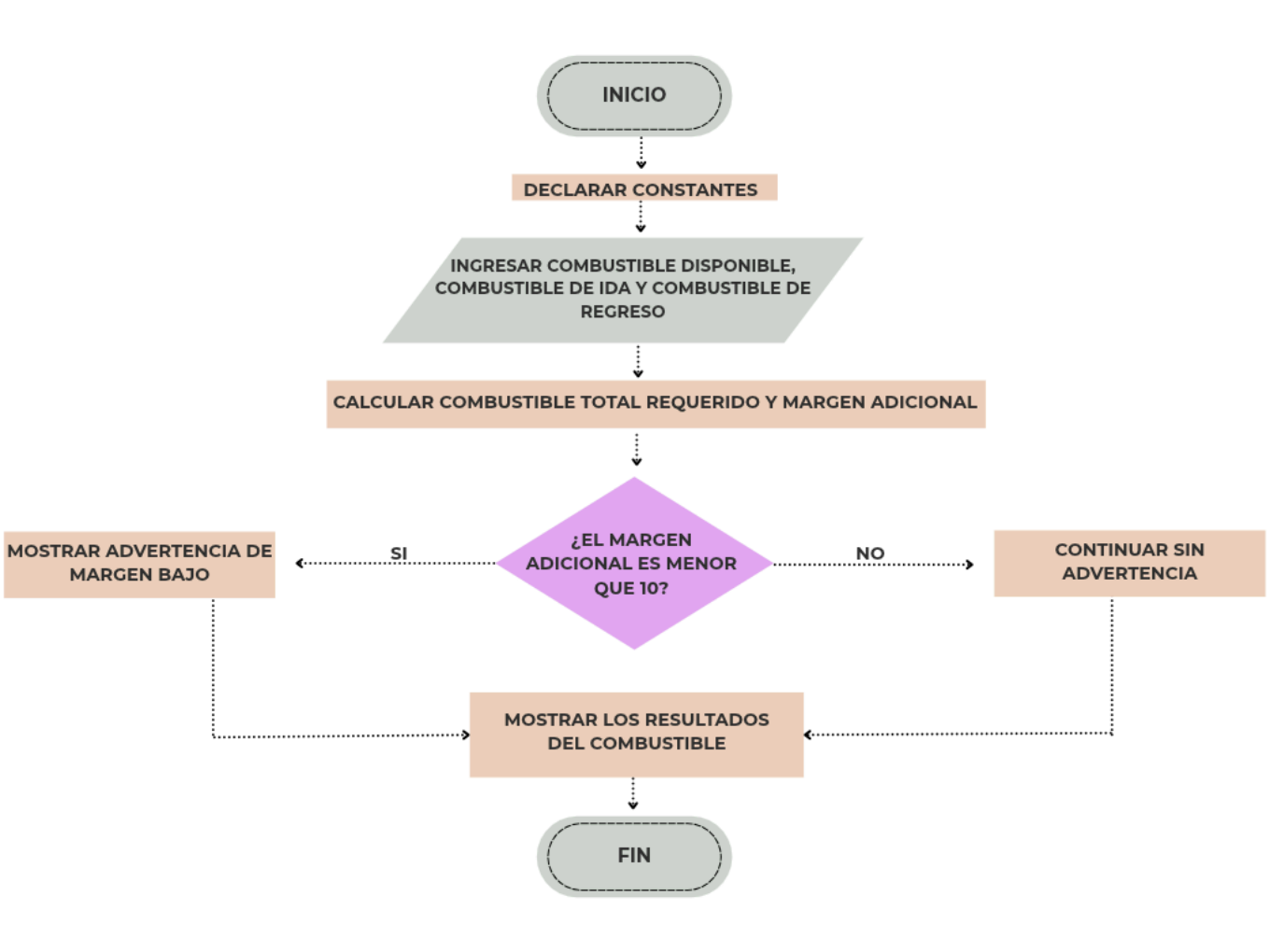
)

print(f"Margen adicional: {margen_adicional} unidades")
```

Tabla de variables

Variable	Descripción	Tipo	Valor o fuente
RESERVA_COMBUSTIBLE	Reserva adicional	Constante entera	10 unidades
MARGEN_MINIMO_COMBUSTIBLE	Límite para la advertencia	Constante entera	10 unidades
combustible_disponible	Combustible disponible	Entero	Entrada del usuario
combustible_ida	Combustible requerido para la ida	Entero	Entrada del usuario
combustible_regreso	Combustible requerido para el regreso	Entero	Entrada del usuario
combustible_total_requerido	Combustible total necesario	Entero	Ida + regreso + reserva
margen_adicional	Diferencia entre disponible y requerido	Entero	Disponible − total requerido

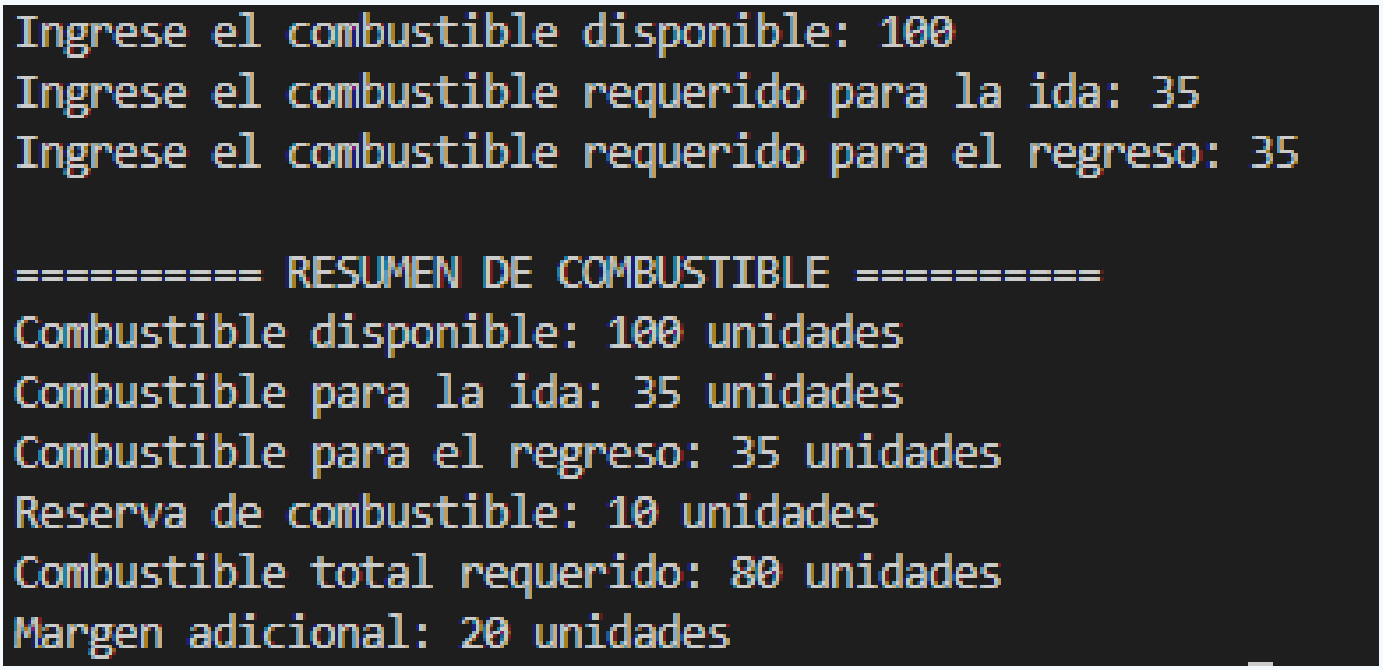
Diagrama de flujo



Evidencias

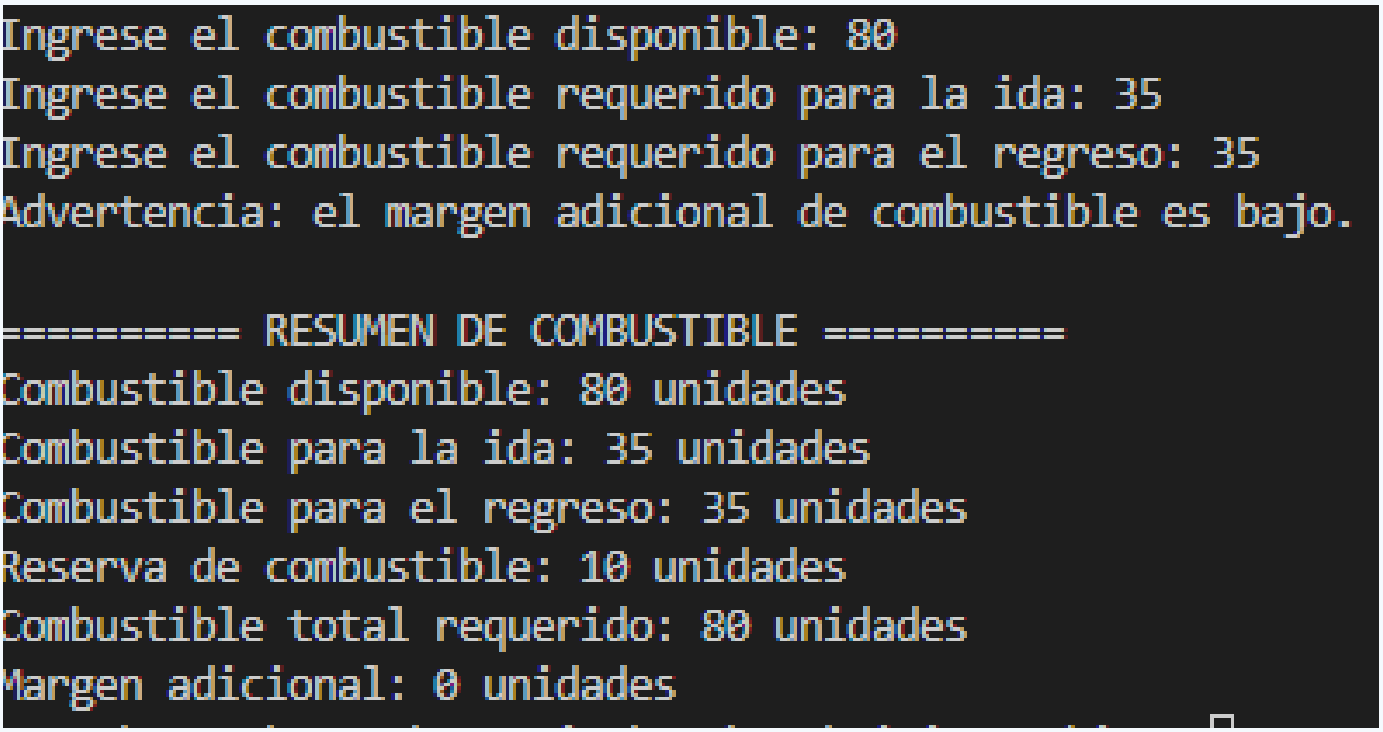
Evidencia CP01

Observación CP01: El combustible disponible es de 100 unidades y el total requerido es de 80. El margen es de 20 unidades, por lo que no se espera una advertencia.



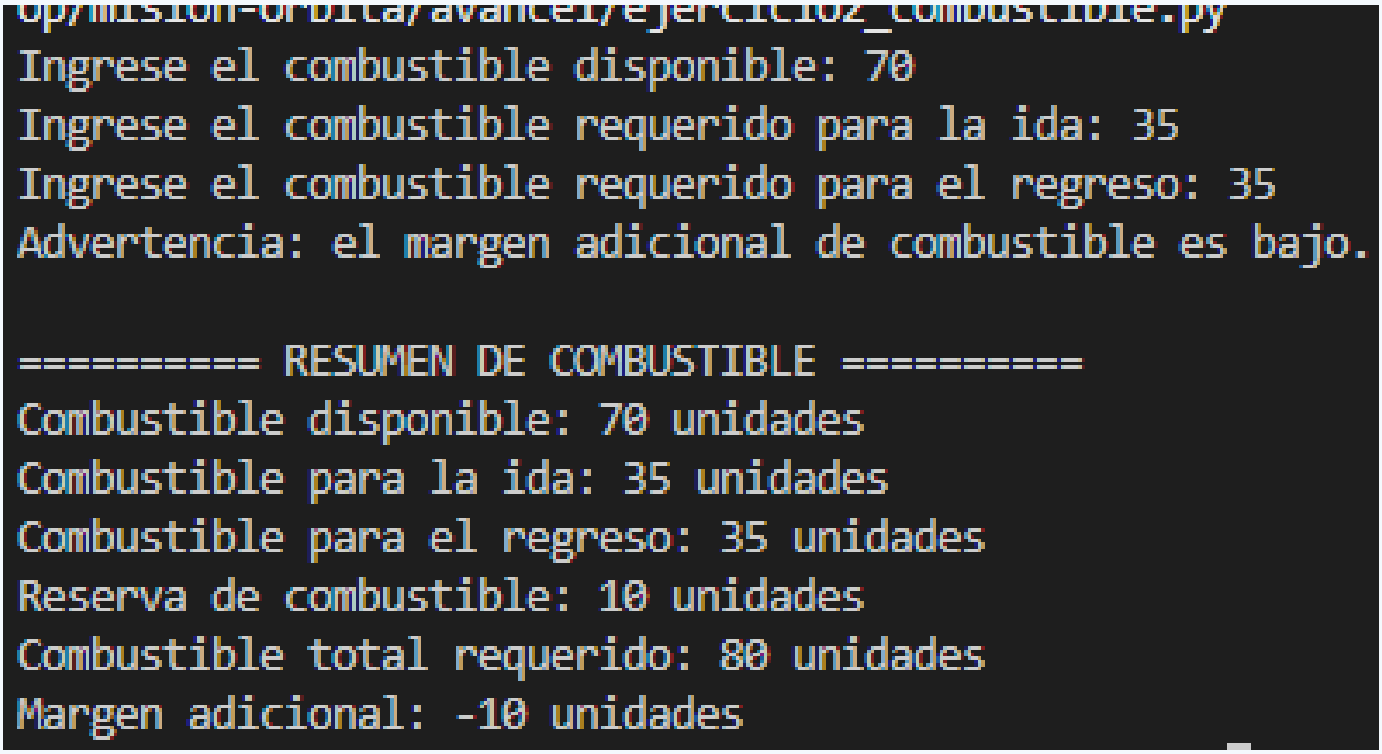
Evidencia CP02

Observación CP02: El combustible disponible es de 80 unidades y el total requerido es de 80. El margen es de 0 unidades, por lo que se espera una advertencia.



Evidencia CP03

Observación CP03: El combustible disponible es de 70 unidades y el total requerido es de 80. El margen es de -10 unidades, por lo que se espera una advertencia.



Observaciones

El programa permite identificar si existe un margen adicional de combustible suficiente. Cuando el margen es inferior a 10 unidades, se muestra la advertencia correspondiente. El cálculo también permite reconocer cuando el combustible disponible no cubre el total requerido.

5.Ejercicio 3 — Soporte de la tripulación

Documentación de la validación de suficiencia de oxígeno y provisiones frente a los valores requeridos.

Código Python

Entrada de datos: solicitar recursos disponibles y requeridos.

```
oxigeno_disponible = int(

input("Ingrese el oxígeno disponible: ")

)

oxigeno_requerido = int(

input("Ingrese el oxígeno requerido: ")

)

provisiones_disponibles = int(

input("Ingrese las provisiones disponibles: ")

)

provisiones_requeridas = int(

input("Ingrese las provisiones requeridas: ")

)
```

Decisión: comprobar si ambos recursos son suficientes.

```
if (

oxigeno_disponible >= oxigeno_requerido

and provisiones_disponibles >= provisiones_requeridas

):

print("La nave posee recursos suficientes para la tripulación.")

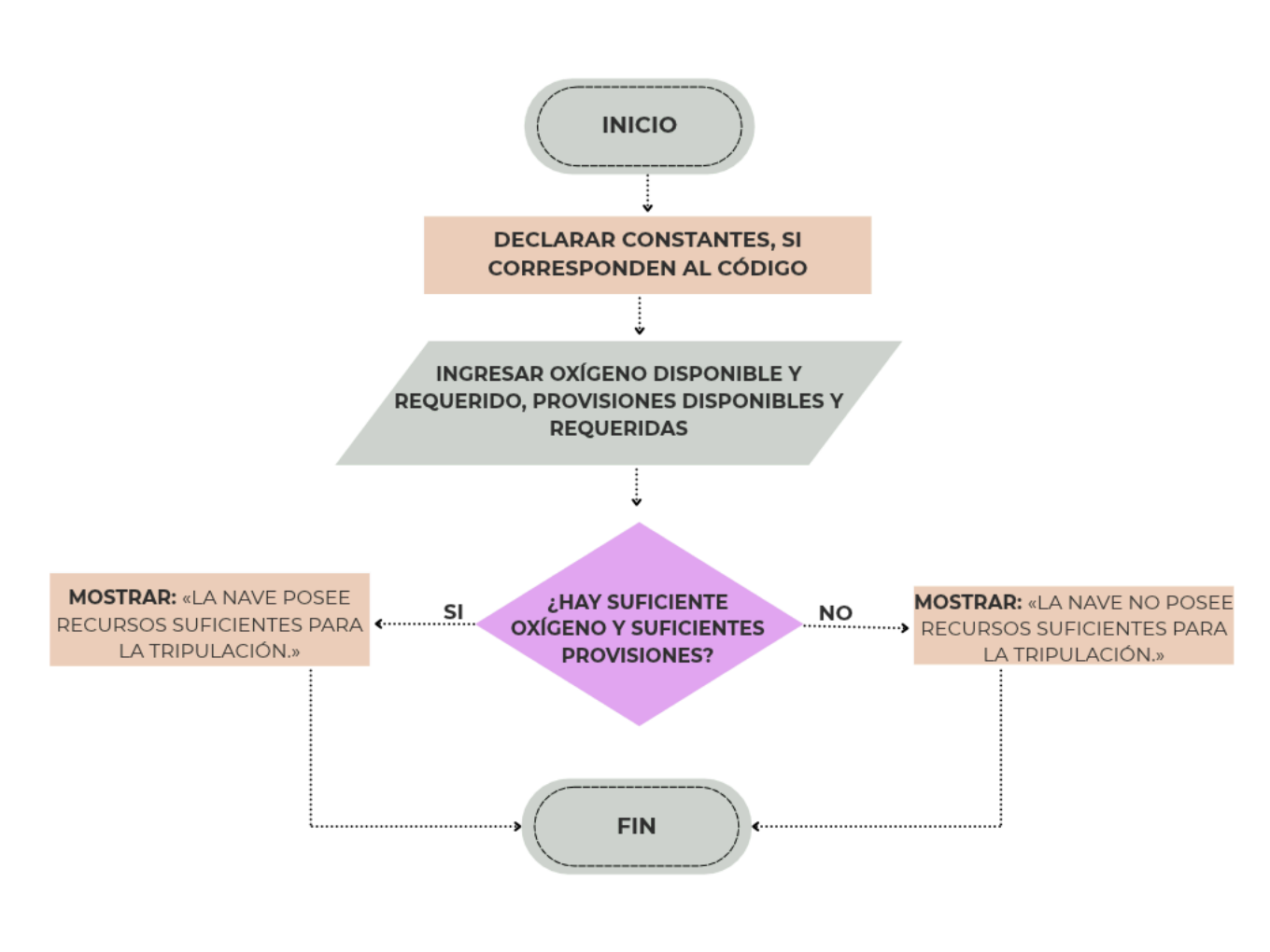
else:

print("La nave no posee recursos suficientes para la tripulación.")
```

Tabla de variables

Variable	Tipo	Descripción
oxigeno_disponible	Entero	Cantidad de oxígeno disponible en la nave.
oxigeno_requerido	Entero	Cantidad de oxígeno necesaria para la misión.
provisiones_disponibles	Entero	Cantidad de provisiones disponibles.
provisiones_requeridas	Entero	Cantidad de provisiones necesaria para la misión.

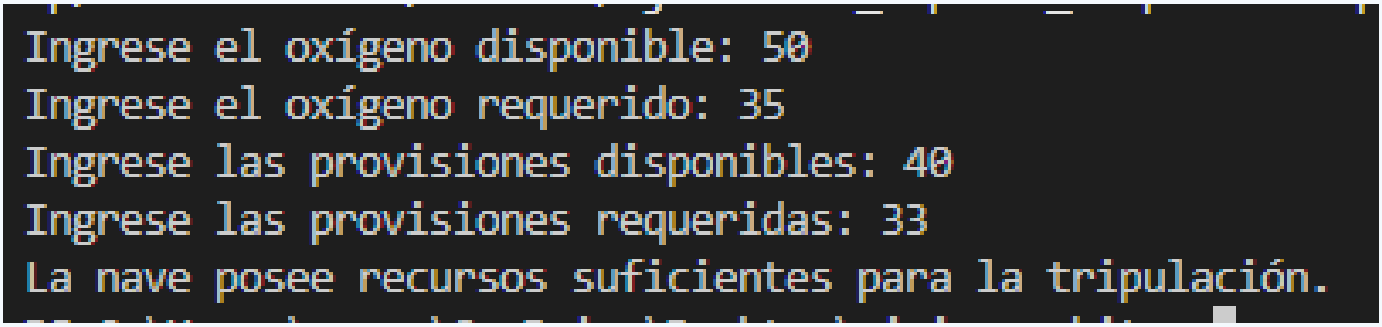
Diagrama de flujo



Evidencias

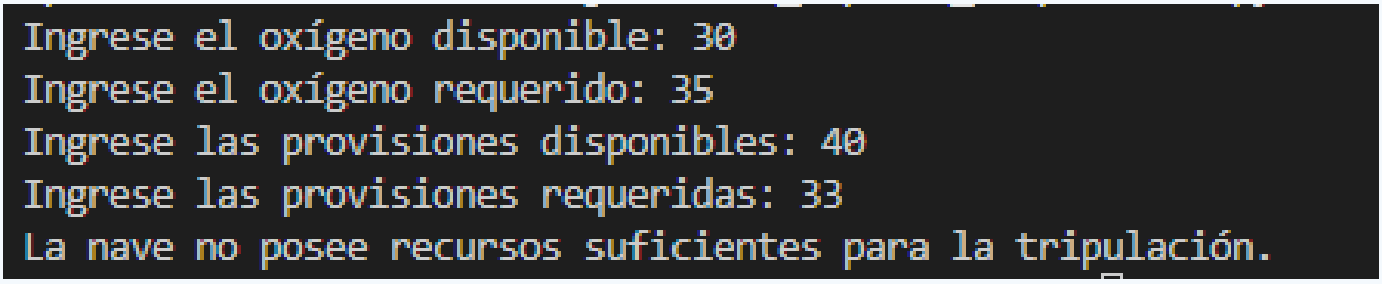
Evidencia CP01

Observación CP01: La prueba verifica que la nave cuenta con oxígeno y provisiones suficientes. Como ambos recursos disponibles son iguales o superiores a los requeridos, el programa muestra el mensaje de recursos suficientes para la tripulación.



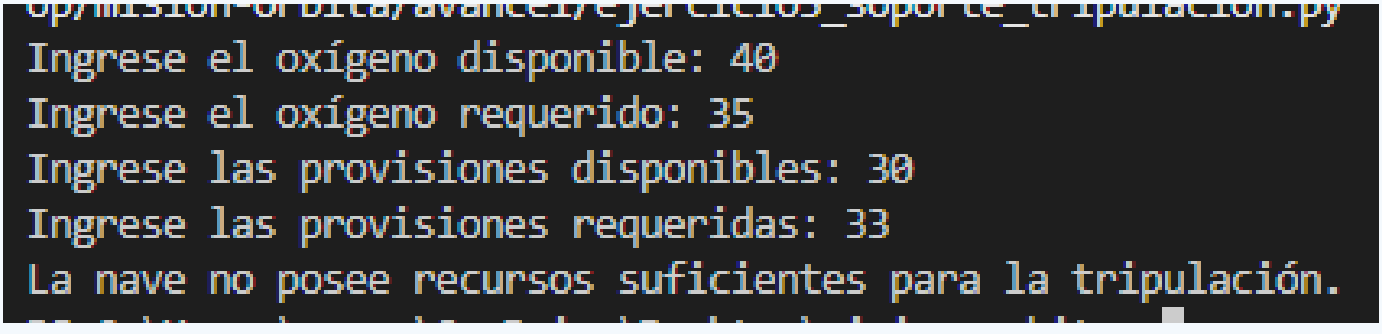
Evidencia CP02

Observación CP02: La prueba verifica un escenario en el que al menos uno de los recursos es insuficiente. El programa muestra el mensaje de que la nave no posee recursos suficientes para la tripulación.



Evidencia CP03

Observación CP03: La prueba comprueba que, cuando el oxígeno y las provisiones disponibles son suficientes para cubrir las cantidades requeridas, el programa autoriza el uso de los recursos para atender a la tripulación.



Observaciones

La estructura condicional comprueba que ambos recursos sean suficientes. Si el oxígeno y las provisiones alcanzan las cantidades requeridas, el programa indica que la nave posee recursos suficientes para la tripulación. Si uno o ambos recursos son insuficientes, muestra el mensaje correspondiente.

6. Ejercicio 4 — Autorización de lanzamiento

Documentación de la decisión de autorización a partir de combustible, oxígeno, energía y provisiones.

Código Python

Declaración de constantes para los cálculos.

```
RESERVA_COMBUSTIBLE = 10
```

Entrada de datos: solicitar los recursos disponibles y los recursos requeridos para la misión.

```
combustible_disponible = int(
input("Ingrese el combustible disponible: ")
)
combustible_ida = int(
input("Ingrese el combustible requerido para la ida: ")
)
combustible_regreso = int(
input("Ingrese el combustible requerido para el regreso: ")
)
oxigeno_disponible = int(
input("Ingrese el oxígeno disponible: ")
)
oxigeno_requerido = int(
input("Ingrese el oxígeno requerido: ")
)
energia_disponible = int(
input("Ingrese la energía disponible: ")
)
energia_requerida = int(
```

```
input("Ingrese la energía requerida: ")                                print(
)
provisiones_disponibles = int(
input("Ingrese las provisiones disponibles: ")
)
provisiones_requeridas = int(
input("Ingrese las provisiones requeridas: ")
)
```

Procesamiento: calcular el combustible total requerido.

```
combustible_total_requerido = (
combustible_ida
+ combustible_regreso
+ RESERVA_COMBUSTIBLE
)
```

Decisión: comprobar si todos los recursos son suficientes.

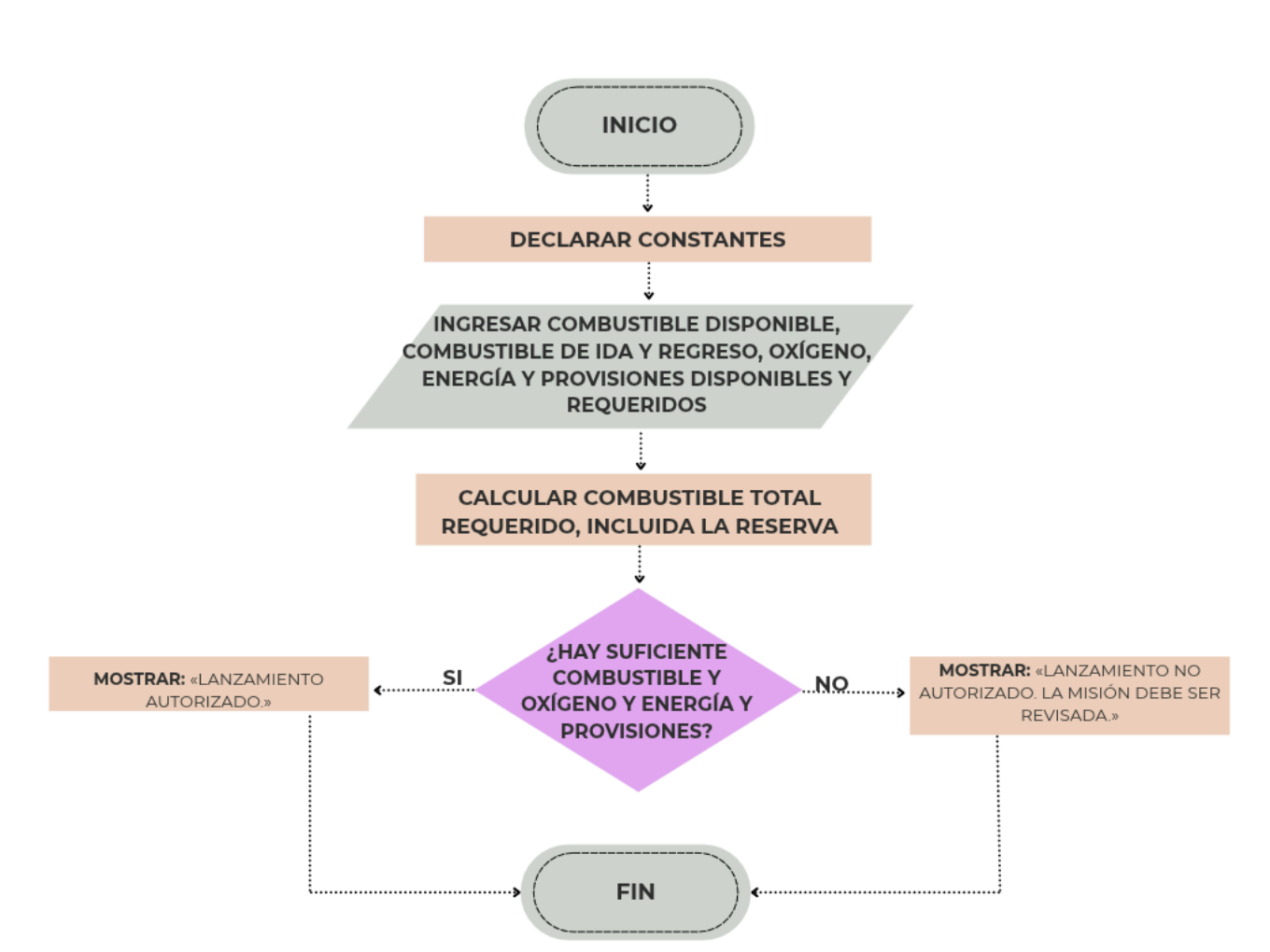
```
if (
combustible_disponible >= combustible_total_requerido
and oxigeno_disponible >= oxigeno_requerido
and energia_disponible >= energia_requerida
and provisiones_disponibles >= provisiones_requeridas
):
print("Lanzamiento autorizado.")
else:
```

Tabla de variables

Variable o constante	Tipo	Descripción
RESERVA_COMBUSTIBLE	Constante entera	Combustible adicional requerido como reserva.
combustible_disponible	Entero	Cantidad de combustible disponible.
combustible_ida	Entero	Combustible requerido para la ida.
combustible_regreso	Entero	Combustible requerido para el regreso.
combustible_total_requerido	Entero	Combustible de ida y regreso más la reserva.
oxigeno_disponible	Entero	Oxígeno disponible en la nave.
oxigeno_requerido	Entero	Oxígeno necesario para la misión.
energia_disponible	Entero	Energía disponible para la misión.
energia_requerida	Entero	Energía necesaria para la misión.
provisiones_disponibles	Entero	Provisiones disponibles para la tripulación.

provisiones_requeridas	Entero	Provisiones necesarias para la tripulación.

Diagrama de flujo



Evidencias

Evidencia CP01

Observación CP03: La prueba comprueba que, cuando el oxígeno y las provisiones disponibles son suficientes para cubrir las cantidades requeridas, el programa autoriza el uso de los recursos para atender a la tripulación.

```
Ingrese el combustible disponible: 100
Ingrese el combustible requerido para la ida: 35
Ingrese el combustible requerido para el regreso: 35
Ingrese el oxígeno disponible: 40
Ingrese el oxígeno requerido: 35
Ingrese la energía disponible: 70
Ingrese la energía requerida: 60
Ingrese las provisiones disponibles: 40
Ingrese las provisiones requeridas: 33
Lanzamiento autorizado.
```

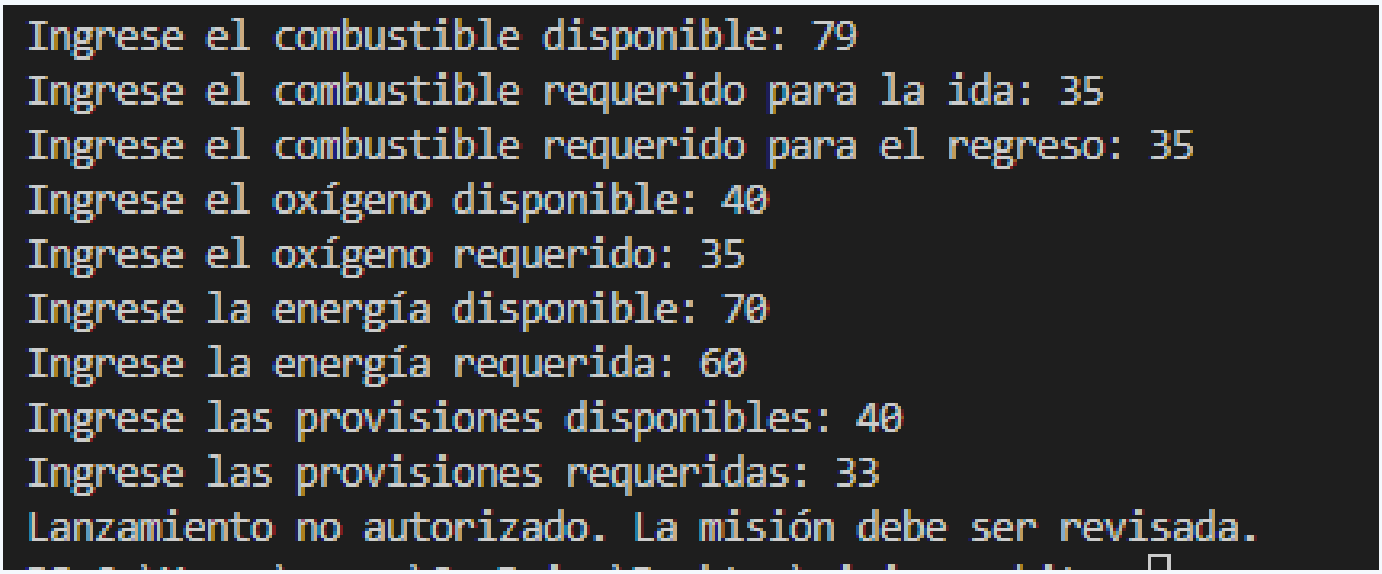
Evidencia CP02

Observación CP03: La prueba comprueba que, cuando el oxígeno y las provisiones disponibles son suficientes para cubrir las cantidades requeridas, el programa autoriza el uso de los recursos para atender a la tripulación.

```
Ingrese el combustible disponible: 100
Ingrese el combustible requerido para la ida: 35
Ingrese el combustible requerido para el regreso: 35
Ingrese el oxígeno disponible: 30
Ingrese el oxígeno requerido: 35
Ingrese la energía disponible: 70
Ingrese la energía requerida: 60
Ingrese las provisiones disponibles: 40
Ingrese las provisiones requeridas: 33
Lanzamiento no autorizado. La misión debe ser revisada.
```

Evidencia CP03

Observación CP03: La prueba verifica nuevamente las condiciones de autorización de la misión. Si todos los recursos cumplen los requisitos, el lanzamiento se autoriza; si alguno es insuficiente, el programa lo rechaza.



Observaciones

La autorización se realiza mediante una condición compuesta que verifica todos los recursos necesarios. Si todas las condiciones se cumplen, el programa muestra «Lanzamiento autorizado». Si al menos una condición falla, muestra el mensaje que indica que la misión debe ser revisada.

7. Conclusión

El desarrollo del avance 1 de Misión Órbita permitió aplicar conceptos fundamentales de programación en Python, como variables, constantes, operaciones aritméticas, estructuras secuenciales y estructuras condicionales.

Los cuatro programas abordan diferentes necesidades de una misión espacial: calcular recursos, evaluar el margen de combustible, verificar las necesidades de la tripulación y determinar si se cumplen las condiciones para autorizar el lanzamiento.

La documentación de los ejercicios mediante tablas de variables, diagramas de flujo y casos de prueba facilita la comprensión de las soluciones y permite comprobar que los resultados obtenidos coincidan con los resultados esperados.

Repositorio GitHub

REPOSITORIO ENLACE

Control de cambios

Versión	Descripción
1.0	Avance 1 — estructura inicial del informe y casos de prueba esperados
Fecha de documentación	9 de octubre de 2026